LAB 01A Introduction

October 26, 2026

1 Programming with Python

Python is a powerful and extensible programming language well-suited to a wide range of tasks. In these sessions you will be introduced to Python as a programming language, and as a tool that can be used to analyse data taken in Engineering experiments. As a programming language it is easy to learn, powerful, and will introduce you to good programming habits. You will also find it a more powerful and flexible way to handle data compared to Excel, with libraries that can handle professional statistical analysis and graph plotting.

1.1 Jupyter Notebooks

Jupyter Notebooks, like this one, have become a very popular tool in technology and data analysis. Notebooks are able to hold a combination of formatted text, specifically Markdown, code, in our case Python, and outputs. Notebooks act like a programming lab book, holding your thoughts, the steps you take in developing code and data analysis, and your results; all in a single place. You will use Jupyter Notebooks as a useful way to keep both code, and results, in an organised fashion.

2 Aims and Objectives

By the end of these sessions you should be able to use Python to

- perform calculations on numbers and arrays of numbers;
- manipulate text and text files;
- use flow control statements such as `for`, `while` and `if` statements;
- extract useful data from complex data structures;
- import standard libraries and use library functions, to avoid reinventing the wheel;
- define and use software functions that you have written yourself;
- design, write, debug, run and test non-trivial programs using an Integrated Development Environment
- use good programming practice, including the use of comments
- plot data and fit simple functions to that data
- undertake simple statistical analysis of experimental data

3 Marking

Each Jupyter Notebook contains one exercise and some associated Markdown and Python cells. Marks are based only on the exercises and the number of marks available is shown for each - other

cells are there to guide you through Python programming and Jupyter Notebooks. *Every time* you complete an exercise show your results to one of the demonstrators to confirm your effort.

4 Introduction

Even if you are already familiar with other programming languages, you should read through this section to familiarize yourself with Python and Jupyter Notebooks, some advice on good programming practice, and information on assessment.

4.1 What is Python?

Python is an interpreted high-level programming language for a wide range of programming tasks. But what does that mean?:

- Interpreted - Python does not require 'compiling', you can run your script as soon as you've written it and you can even run small snippets directly in a Python console.
- High-level - the code you write in Python is relatively human-readable and very different to the instructions received by the computer when your code runs.
- Programming - using a set of instructions for computers to create algorithms that complete a task, usually to make our lives more efficent.

Guido van Rossum released the first version of Python in 1991. Python's philosophy of readability and simplicity has made it an extremely popular programming language. Python is used for research, teaching and data science across the globe and some of its more famous users include Wikipedia, Google, NASA and Reddit (which is almost entirely written in Python). Unlike some other coding systems, such as MATLAB or R, Python has not been designed with a particular use case in mind; instead, Python is heavily extensible through modules that have been designed for specific tasks. By combining modules together Python can be used to easily compute anything that can be done in other languages in a way that is both quick to code and quick to run. Beyond modules for analysis and processing, there are many modules for one-, two- and three- dimensional plotting. During the labs you will be expected to use a range of these modules.

4.2 What are Jupyter Notebooks?

Short calculations can be executed directly from a Python console/terminal, similar to what you would do on a pocket calculator. However, more extensive calculations, or procedures that you want to repeat several times, are more conveniently programmed in short scripts (often saved as `.py` files). In scientific computing and data science we often want more than just a script, we want to combine our thoughts and assumptions, codes, outputs and conclusions and we want to do this in a way that's easy to share and reproduce.

Enter Jupyter Notebooks (formerly IPython Notebooks) a web-based interactive environment for creating 'notebooks' that contain cells of information. Each cell can be text (using Markdown), code (Python), mathematics (using the markup language LaTeX), plots and other media. Each notebook is saved as a `.ipynb` file, which is easy to send to a collaborator should they need to see the steps taken to get from A to B.

Here at the University of Glasgow, we're running a 'Jupyter Hub', a multi-user server that allows each of you to have your own Jupyter Notebook environment for completing the programming unit, for use throughout the labs, and future applications throughout your studies at the University of

Glasgow. We strongly encourage you to create a new notebook for each practical experiment you do - all your data analysis can be compeleted using Python.

5 Assessment

Your Python exercises are assessed continuously during the sessions. Once you have completed one of the exercises, show and explain it to one of the demonstrators, who will then enter your mark on the marking sheet. You can also talk to demonstrators if you get stuck or if you want to discuss any aspects of your programming. They will be able to give you advice on good programming style, how to write effective and clear code, and how to go about testing and tracking down any errors in your code. There is also no problem about disussing your programming efforts and challenges with your peers - although you will have to explain what ***you*** have done to the lab demonstrators for marking. Marks are for individuals.

Make sure you have all your work marked and signed for as soon as possible in order to avoid a build up of unmarked material at the end of the sessions - labs finish at midday, and demonstrators will not stay longer for additional marking.

6 Saving Your Work

Where an exercise asks you to save a plot, you should do so as a `.png` file alongside your `.ipynb` file containing the Python code and associated texts and outputs that generated it. That way there is a permanent record of what you have done, and you will not have to re-create the graph from scratch if a demonstrator wants to see it again.

7 Help and Documentation

Python is more than just a programming language, it's a huge combination of possible modules and community-built functions and tools. You can't learn everything about it at once or always remember how you have done things before - but fortunately Python has a huge community and the modules we will use have excellent documentation and so you don't have to!

Five core documentations should give you all the technical answers you need during the course:

- Python: https://docs.python.org/3/
- NumPy: https://docs.scipy.org/doc/numpy/
- SciPy: https://docs.scipy.org/doc/scipy/reference/
- Matplotlib: https://matplotlib.org/
- Jupyter Notebooks: https://jupyter-notebook.readthedocs.io/en/stable/index.html

You will also want to make use of your favourite search engine and a community website called Stack Overflow, e.g. `https://stackoverflow.com/questions/tagged/python`. Of course, if you can't find what you need you can ask a demonstrator.

8 Commenting

It is good practice to comment your codes so the reader (which might be you when you next look at it) knows how you arrived at your answer, or how to interpret the result. In a Jupyter Notebook

you should do this in two ways:

1. By using a Markdown cell (such as this one) to write a chunk of formatted text. Markdown cells can use rich formatting, HTML and LaTeX allowing you to include images, videos, formulae and maths. Throughout the course you will be asked questions which you should answer in cells like this.
2. By commenting the Python code itself (see below).

In the top panel you can see that this cell is a Markdown cell. You can switch between different styles by selecting e.g. "Code" from the drop-down menu. If you want to return it to "Markdown" make your selection and run the cell by pressing `Ctrl+Enter`.

8.1 Python Comments

In Python, everything after a `#` will be treated as a comment and not run when the rest of the code is, for example,

```
x = 2  # assigns the value of 2 to x
```

That was not a very informative comment, though (it should hopefully be obvious that that is what is happening!). Typically a comment will describe the *purpose* of one or more lines of code. For example,

```
r = np.sqrt(x**2 + y**2)  # use Pythagoras to calculate radial distance
```

If you hit `Ctrl+Enter` in the cell below, it should run the code and produce a result.

[ ]:
```
import numpy as np
x = 3
y = 4
r = np.sqrt(x**2 + y**2)
print(r)
```